# Database – Lab 1
# Adhm ElShabasy – 23010270

This project designs ER models from an English written description, it runs through a Gemini API key that translates it into a pre-designed json format, which in order goes through the python code powered by Graphviz to turn it into an ER diagram.

## 1- Structure

It consists of three parts:

1. render.py
2. generate.py
3. prompt.txt

it runs in the format: python generate.py <description.txt> <output.json>, and saves the json file and the ER model as a png under the name diagram
The API key has no place in the code but rather is put in the terminal before conducting work as not to need to change it ($env:GEMINI_API_KEY = "key" )

## 2- JSON format and prompt

here is the part of the prompt given to the API which include the json format, I didn't include the whole thing as it was too long, it needed a lot of explanation as the lack of details could lead to misinterpretation:

```
{
 "entities": [
 {
 "name": "<unique string>",
 "isWeak": true,
 "attributes": [ <attribute>, ... ]
 }
 ],
 "relationships": [ <relationship>, ... ]
}
```

An <attribute> is an object. "name" is required, the rest are optional booleans/lists:

```
{
 "name": "<string>",
 "isPrimaryKey": true,        key that identifies a regular (strong) entity
 "isPartialKey": true,        discriminator of a WEAK entity (identifies it only together with its owner)
 "isForeignKey": true,        attribute that stores the key of another entity
 "isDerived": true,           value can be computed from other data (e.g. Age from BirthDate)
 "isMultiValued": true,       can hold several values for one instance (e.g. Phone)
 "composite": ["<part>", "<part>"]   made of named parts (e.g. Name -> FirstName, LastName); a list of plain strings
}
```

Flags can be combined where it makes sense (a composite primary key, a multivalued composite attribute, a composite partial key, a derived multivalued attribute).

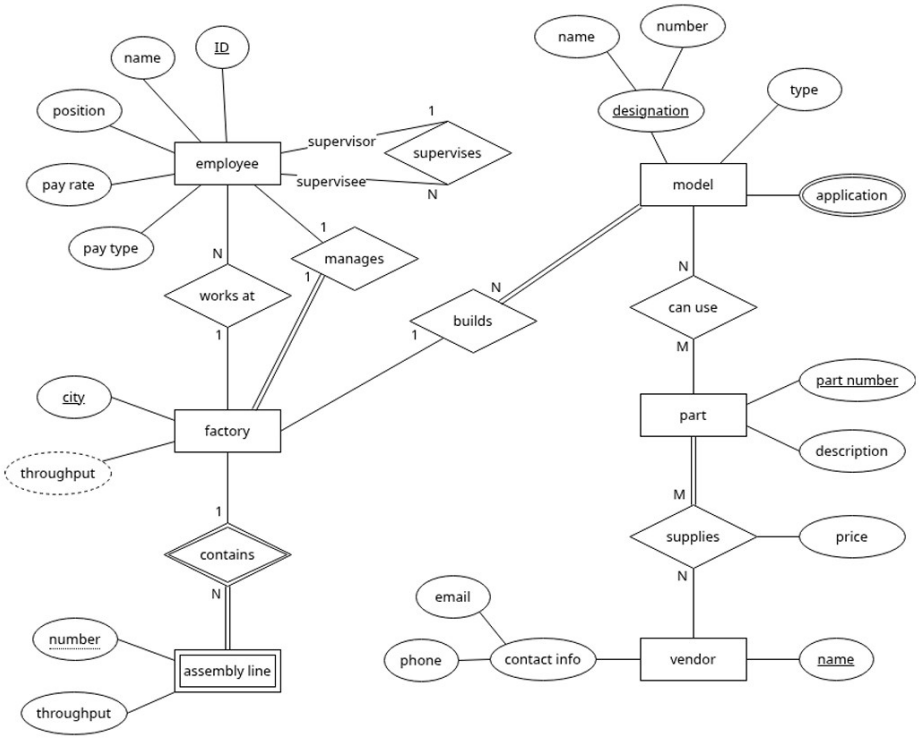
A <relationship> has ONE of two shapes.

Shape A - exactly two participants (this includes recursive relationships):
{
  "entity1": "<entity name>",
  "entity2": "<entity name>",
  "name": "<string>",
  "cardinality": "<part1>:<part2>",
  "participation1": "total",  (optional, only "total")
  "participation2": "total",  (optional, only "total")
  "role1": "<string>",  (optional)
  "role2": "<string>",  (optional)
  "isIdentifying": true,  (optional)
  "attributes": [ <attribute>, ... ] (optional, attributes of the relationship itself)
}

Shape B - three or more participants (n-ary relationship):
{
  "name": "<string>",
  "participants": [
    { "entity": "<entity name>", "cardinality": "<string>", "participation": "total", "role": "<string>" },
    ...
  ],
  "isIdentifying": true,  (optional)
  "attributes": [ <attribute>, ... ] (optional)
}

## 3- Example

Here is an example from the lectures, which I turned into a description and gave to my program so we can compare the output:



here is its description:
A manufacturing company keeps data about its factories, employees, models, parts and vendors.

Each employee has a unique ID, a name, a position, a pay rate and a pay type. An employee works at exactly one factory, and a factory has many employees. Each employee may supervise other employees; an employee has at most one supervisor (the roles are supervisor and supervisee).

Each factory is identified by its city. It also has a throughput, which is calculated from the throughput of its assembly lines. Every factory must be managed by exactly one employee, and an employee manages at most one factory.

A factory contains many assembly lines. An assembly line cannot exist without its factory: it is identified within its factory by a number, and it also has a throughput. Every assembly line belongs to exactly one factory.

A model is identified by its designation, which is made of a name and a number. A model also has a type, and it can have several applications. Every model must be built by exactly one factory, while a factory builds many models.

A part has a unique part number and a description. A model can use many parts, and a part can be used by many models.

A vendor is identified by its name and has contact info made of an email and a phone number. A vendor supplies many parts and a part is supplied by many vendors; every part must have at least one vendor. For each supply we record the price.

Here is the output json:

```
{
  "entities": [
    {
      "name": "Employee",
      "attributes": [
        {
          "name": "ID",
          "isPrimaryKey": true
        },
        {
          "name": "Name"
        },
        {
          "name": "Position"
        },
        {
          "name": "PayRate"
        },
        {
          "name": "PayType"
        }
      ]
```

```
},
{
  "name": "Factory",
  "attributes": [
    {
      "name": "City",
      "isPrimaryKey": true
    },
    {
      "name": "Throughput",
      "isDerived": true
    }
  ]
},
{
  "name": "AssemblyLine",
  "isWeak": true,
  "attributes": [
    {
      "name": "Number",
      "isPartialKey": true
    },
    {
      "name": "Throughput"
    }
  ]
},
{
  "name": "Model",
  "attributes": [
    {
      "name": "Designation",
      "isPrimaryKey": true,
      "composite": [
        "Name",
        "Number"
      ]
    },
    {
      "name": "Type"
    },
    {
      "name": "Application",
      "isMultiValued": true
    }
  ]
},
{
  "name": "Part",
  "attributes": [
    {
      "name": "PartNumber",
      "isPrimaryKey": true
    },
    {
```

```
        "name": "Description"
      }
    ]
  },
  {
    "name": "Vendor",
    "attributes": [
      {
        "name": "Name",
        "isPrimaryKey": true
      },
      {
        "name": "ContactInfo",
        "composite": [
          "Email",
          "Phone"
        ]
      }
    ]
  }
],
"relationships": [
  {
    "entity1": "Employee",
    "entity2": "Factory",
    "name": "Works_At",
    "cardinality": "N:1",
    "participation1": "total"
  },
  {
    "entity1": "Employee",
    "entity2": "Employee",
    "name": "Supervises",
    "cardinality": "1:N",
    "role1": "supervisor",
    "role2": "supervisee"
  },
  {
    "entity1": "Factory",
    "entity2": "Employee",
    "name": "Manages",
    "cardinality": "1:1",
    "participation1": "total",
    "participation2": "total"
  },
  {
    "entity1": "Factory",
    "entity2": "AssemblyLine",
    "name": "Contains",
    "cardinality": "1:N",
    "participation2": "total",
    "isIdentifying": true
  },
  {
    "entity1": "Factory",
```

```
      "entity2": "Model",
      "name": "Builds",
      "cardinality": "1:N",
      "participation2": "total"
    },
    {
      "entity1": "Model",
      "entity2": "Part",
      "name": "Uses",
      "cardinality": "M:N"
    },
    {
      "entity1": "Vendor",
      "entity2": "Part",
      "name": "Supplies",
      "cardinality": "M:N",
      "participation2": "total",
      "attributes": [
        {
          "name": "Price"
        }
      ]
    }
  ]
}
```

And here the output diagram:

